

PULSO URBANO

Modelo de Dados

Para que serve cada tabela, coluna e rotina

PostgreSQL · 9 tabelas · gatilho, funções e views de operação

Squad Elementares · FATEC São José dos Campos · DSM 4º semestre

Cliente: Tecsus · Região piloto: São José dos Campos

Setembro de 2026

Resumo. Este documento explica o modelo de dados relacional do Pulso Urbano, definido no script `modelo_completo_postgres.sql`. Para cada tabela, mostra para que ela serve, quem escreve e quem lê, o significado de cada coluna e as regras que protegem os dados. No fim, descreve as rotinas de manutenção e os pontos que ainda precisam ser alinhados entre o script, as decisões do squad e o código atual.

Ideia central: o banco guarda o histórico ambiental do município de forma permanente e confiável. Nada é apagado fisicamente, toda leitura aponta para o sensor que a gerou, e todo sensor aponta para a estação e, por ela, para o município que é dono do dado.

1. Visão geral

O modelo tem 9 tabelas, organizadas em quatro blocos. Cada bloco atende uma parte do produto e é responsabilidade de microsserviços diferentes.

Bloco	Tabelas	Para que serve
Acesso e Identidade	papeis, usuarios	Quem pode entrar na plataforma, com qual perfil e em qual município (US-01 e US-02).
Domínio Operacional	estacoes, sensores, medidas, regras_alerta, alarmes	O inventário de estações e sensores, a série histórica calibrada, as regras de alerta e os incidentes gerados (US-03 a US-06).
Banco Temporário (Staging)	staging_telemetria	Guarda a leitura bruta no momento em que chega do MQTT, antes de qualquer processamento.
Histórico Agregado	medidas_resumo_diario	Resumo diário por sensor (mínimo, máximo, média), para relatórios de longo prazo sem varrer o volume bruto.

O caminho de uma leitura no banco

- A estação publica a leitura bruta; o ingestao-dados grava o payload em staging_telemetria.
- O servico-validacao lê a mensagem pendente, busca o fator e o ganho do sensor em sensores e calcula **Valor_real = (Valor_bruto × fator) + ganho**.
- O valor calibrado é gravado em medidas e a mensagem de staging é marcada como processada.
- O mesmo serviço compara o valor com as regras_alerta ativas daquele sensor. Se houver violação, grava um registro em alarmes.
- Painéis e relatórios leem medidas, alarmes e medidas_resumo_diario, sempre filtrando pelo município da estação.

2. Diagrama entidade-relacionamento

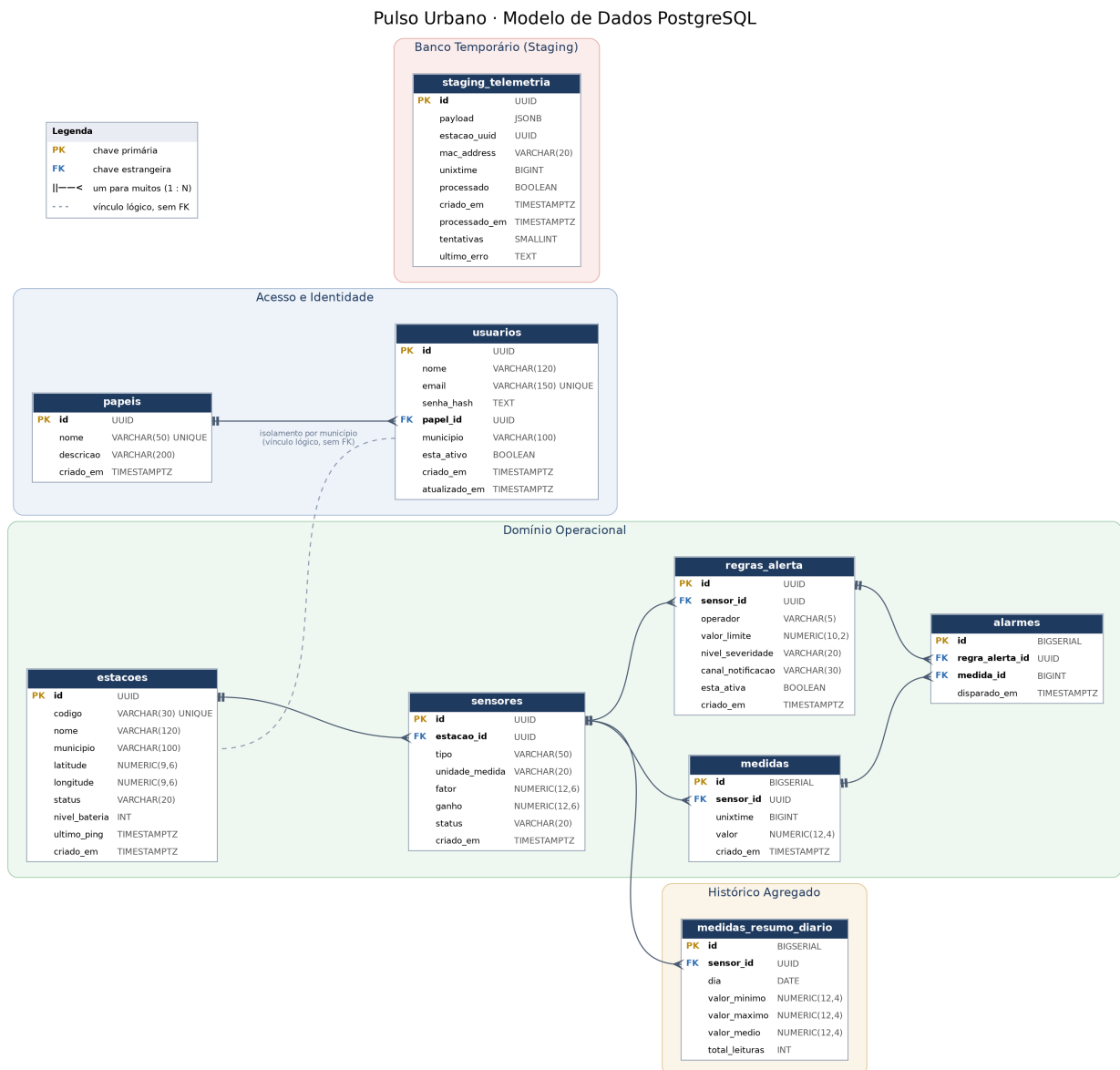


Figura 1 · Tabelas, colunas e relacionamentos do modelo. As cores indicam o bloco de cada tabela. A imagem em alta resolução está em docs/modelo_er.png.

3. Como as tabelas se relacionam

Relação	Cardinalidade	O que significa
papeis → usuarios	1 : N	Cada usuário tem exatamente um papel; um papel é usado por vários usuários.
estacoes → sensores	1 : N	Todo sensor pertence a uma estação. Uma estação tem vários sensores, no máximo um por grandeza.
sensores → medidas	1 : N	Cada leitura calibrada aponta para o sensor que a gerou.
sensores → regras_alerta	1 : N	As regras de alerta são configuradas por sensor.
regras_alerta → alarmes	1 : N	Uma regra pode gerar vários incidentes ao longo do tempo.
medidas → alarmes	1 : N	Uma leitura pode violar mais de uma regra, mas cada par regra + leitura gera um único alarme.
sensores → medidas_resumo_diario	1 : N	Um resumo por sensor por dia.
usuarios ↔ estacoes	lógica	Não há chave estrangeira: o vínculo é o campo municipio . O município vem do token JWT e filtra as estações que o usuário enxerga.

Por que todas as chaves estrangeiras usam ON DELETE RESTRICT

O banco se recusa a apagar um registro que ainda é referenciado. Isso impede, por exemplo, que a exclusão de uma estação apague em cascata os sensores e anos de medições. Combinado com a regra de negócio de **inativação lógica** (campos de status e esta_ativo), garante que o histórico nunca se perca.

Por que o staging não tem chave estrangeira

staging_telemetria é propositalmente isolada. A leitura precisa ser aceita mesmo que a estação ainda não esteja cadastrada ou que o banco relacional esteja sob carga: primeiro se guarda, depois se valida. Isso atende diretamente a prioridade do cliente de nunca perder uma leitura de campo.

4. Tabela a tabela

4.1 papeis

Catálogo fixo dos perfis de acesso. O script já insere os três papéis do produto, e o nome do papel é o valor que vai na claim papel do token JWT.

Quem escreve: script de criação (seed)

Quem lê: gestao-usuarios, auth-service

Coluna	Tipo	Regras	Para que serve
id	UUID	PK · gerado automaticamente	Identificador do papel.
nome	VARCHAR(50)	obrigatório · único	Nome canônico: ADMINISTRADOR, GESTOR_PUBLICO ou PESQUISADOR.
descricao	VARCHAR(200)	opcional	Texto explicativo exibido no cadastro de usuários.
criado_em	TIMESTAMPZ	padrão: agora	Auditoria: quando o papel foi criado.

Os três papéis: ADMINISTRADOR (gestão global da plataforma), GESTOR_PUBLICO (opera e monitora só o próprio município) e PESQUISADOR (consulta às séries históricas). A inserção usa ON CONFLICT DO NOTHING, então rodar o script de novo não duplica os papéis.

4.2 usuarios

Contas de acesso à plataforma. Guarda a identidade, o perfil e o território de cada pessoa.

Quem escreve: gestao-usuarios

Quem lê: auth-service (login) e gestao-usuarios

Coluna	Tipo	Regras	Para que serve
id	UUID	PK	Identificador do usuário; vai na claim id do token.
nome	VARCHAR(120)	obrigatório	Nome exibido na interface.
email	VARCHAR(150)	obrigatório · único	Login do usuário. Não pode haver dois usuários com o mesmo e-mail.
senha_hash	TEXT	obrigatório	Hash bcryptjs com 10 salt rounds. A senha nunca é guardada em texto.
papel_id	UUID	FK → papeis · obrigatório	Perfil de acesso do usuário.
municipio	VARCHAR(100)	obrigatório para gestor (gatilho)	Território do usuário; vai na claim municipio. Vazio para o administrador, que tem visão global.
esta_ativo	BOOLEAN	padrão: TRUE	Inativação lógica: FALSE bloqueia o login sem apagar o registro.
criado_em	TIMESTAMPZ	padrão: agora	Auditoria: data de criação da conta.
atualizado_em	TIMESTAMPZ	padrão: agora	Auditoria: última alteração da conta.

Regra de município: o PostgreSQL não permite consultar outra tabela dentro de um CHECK, então a exigência de município para GESTOR_PUBLICO é feita pelo gatilho trg_valida_municipio_gestor (seção 5).

Índices: idx_usuarios_municipio acelera a listagem por município; idx_usuarios_ativos é parcial e guarda só o e-mail de contas ativas, que é o que o login consulta.

4.3 estacoes

Inventário das estações meteorológicas. É a entidade central do domínio: sem estação cadastrada, não existe fonte de dado.

Quem escreve: gestao-estacoes

Quem lê: gestao-estacoes, analise-dados e demais serviços de consulta

Coluna	Tipo	Regras	Para que serve
id	UUID	PK	Identificador interno da estação.
codigo	VARCHAR(30)	obrigatório · único	Código alfanumérico, imutável após a criação (ex.: EST-SJC-001).
nome	VARCHAR(120)	obrigatório	Nome de exibição da estação.
municipio	VARCHAR(100)	obrigatório	Município dono da estação. É a base do isolamento territorial.
latitude	NUMERIC(9,6)	entre -90 e 90	Posição geográfica, com precisão de cerca de 10 cm.
longitude	NUMERIC(9,6)	entre -180 e 180	Posição geográfica, com precisão de cerca de 10 cm.
status	VARCHAR(20)	Ativa · Inativa · Com Falha	Status administrativo. "Inativa" é a exclusão lógica e prevalece sobre o sinal.
nivel_bateria	INT	0 a 100 · opcional	Saúde do hardware em campo, em porcentagem.
ultimo_ping	TIMESTAMPTZ	opcional	Última comunicação recebida. Vazio indica que a estação nunca transmitiu. Base do monitoramento de disponibilidade (US-04).
criado_em	TIMESTAMPTZ	padrão: agora	Auditoria: data do cadastro.

Índices: idx_estacoes_municipio para o filtro territorial presente em quase toda consulta; idx_estacoes_ultimo_ping para o painel de disponibilidade, que pergunta quais estações estão sem sinal.

4.4 sensores

Sensores instalados em cada estação e seus parâmetros de calibração. É aqui que o sistema aprende a converter a leitura bruta em grandeza real, o que permite aceitar qualquer sensor por configuração.

Quem escreve: gestao-estacoes

Quem lê: servico-validacao (calibração) e gestao-estacoes

Coluna	Tipo	Regras	Para que serve
id	UUID	PK	Identificador do sensor.
estacao_id	UUID	FK → estacoes · obrigatório	Estação onde o sensor está instalado.
tipo	VARCHAR(50)	lista fechada (CHECK)	Grandeza medida (ex.: Temperatura, PM2.5). Ver seção 8.
unidade_medida	VARCHAR(20)	obrigatório	Unidade do valor calibrado (°C, %, µg/m³, km/h...).
fator	NUMERIC(12,6)	padrão: 1.0	Multiplicador da calibração. 1.0 significa sensor já calibrado.
ganho	NUMERIC(12,6)	padrão: 0.0	Deslocamento (offset) da calibração. 0.0 significa sem compensação.
status	VARCHAR(20)	Ativo · Inativo	Inativação lógica, preservando as medições já gravadas.
criado_em	TIMESTAMPTZ	padrão: agora	Auditoria: data do cadastro.

Unicidade: a restrição uq_sensor_por_estacao (estacao_id, tipo) impede dois sensores da mesma grandeza na mesma estação; a API devolve 409 nesse caso.

Calibração: $\text{Valor_real} = (\text{Valor_bruto} \times \text{fator}) + \text{ganho}$. Com os valores padrão, o valor bruto passa inalterado.

4.5 medidas

A série histórica: cada linha é uma leitura já calibrada de um sensor. É a tabela que mais cresce e a mais consultada pelos painéis e relatórios. Por decisão do cliente, é mantida enquanto o projeto existir.

Quem escreve: servico-validacao

Quem lê: analise-dados, servico-relatorios

Coluna	Tipo	Regras	Para que serve
id	BIGSERIAL	PK sequencial	Identificador da leitura. Inteiro sequencial indexa melhor que UUID numa tabela grande e lida em ordem de tempo.
sensor_id	UUID	FK → sensores · obrigatório	Sensor que gerou a leitura.
unixtime	BIGINT	obrigatório	Momento da leitura na estação, em segundos desde 1970 (UTC). BIGINT evita o estouro de 2038 dos inteiros de 32 bits.
valor	NUMERIC(12,4)	obrigatório	Valor já calibrado. A leitura bruta fica só no staging.

Coluna	Tipo	Regras	Para que serve
criado_em	TIMESTAMPTZ	padrão: agora	Momento em que a leitura foi gravada no servidor. A diferença para o unixtime mostra atraso de ingestão.

Índice: `idx_medidas_sensor_tempo` (`sensor_id`, `unixtime` DESC) atende a consulta típica de gráfico: leituras de um sensor num período, da mais recente para a mais antiga.

4.6 regras_alerta

Regras cadastradas pelo gestor para um sensor. A regra fica inerte aguardando medições; ela não gera dado sozinha.

Quem escreve: API de regras de alerta

Quem lê: `servico-validacao` (a cada leitura) e a interface de alertas

Coluna	Tipo	Regras	Para que serve
id	UUID	PK	Identificador da regra.
sensor_id	UUID	FK → sensores · obrigatório	Sensor monitorado pela regra.
operador	VARCHAR(5)	> · < · >= · <= · =	Comparação aplicada entre a leitura e o limite.
valor_limite	NUMERIC(10,2)	obrigatório	Limiar, na mesma unidade do sensor (ex.: 50.00 mm de chuva).
nivel_severidade	VARCHAR(20)	Informativo · Alerta · Critico	Gravidade do incidente; define o badge de risco na interface.
canal_notificacao	VARCHAR(30)	Painel · Email · SMS	Canal pelo qual o aviso deve ser enviado.
esta_ativa	BOOLEAN	padrão: TRUE	Inativação lógica: regra inativa deixa de gerar alarmes.
criado_em	TIMESTAMPTZ	padrão: agora	Auditoria: data do cadastro.

Índices: `idx_regras_ativas_por_sensor` é parcial e guarda só as regras ativas. É o caminho mais usado do sistema, porque o `servico-validacao` consulta as regras ativas a cada leitura recebida. `idx_regras_sensor` e `idx_regras_criado_em` atendem listagens e filtros da interface.

4.7 alarmes

Os incidentes: cada linha registra que uma leitura real violou uma regra ativa. É o histórico oficial de alertas.

Quem escreve: servico-validacao (único serviço que escreve)

Quem lê: alertas-notificacoes, relatórios e painéis

Coluna	Tipo	Regras	Para que serve
id	BIGSERIAL	PK sequencial	Identificador do incidente.
regra_alerta_id	UUID	FK → regras_alerta · obrigatório	Regra que foi violada.
medida_id	BIGINT	FK → medidas · obrigatório	Leitura que causou a violação, com o valor e o horário exatos.
disparado_em	TIMESTAMPTZ	padrão: agora	Momento em que o incidente foi registrado.

Um incidente por violação: a restrição `uq_alarme_por_medida_regra` (`regra_alerta_id`, `medida_id`) impede alarmes duplicados quando a mesma mensagem é reprocessada, por exemplo numa reentrega do RabbitMQ.

4.8 staging_telemetria

Zona de amortecimento entre o MQTT e o processamento. Guarda a mensagem exatamente como chegou, para que nenhuma leitura se perca se o processamento falhar. É a parte PostgreSQL do banco temporário; o Redis atua como buffer rápido na frente dele.

Quem escreve: ingestao-dados

Quem lê: servico-validacao (lê e marca como processada)

Coluna	Tipo	Regras	Para que serve
id	UUID	PK	Identificador da mensagem.
payload	JSONB	obrigatório	Mensagem bruta do dispositivo (<code>uuid</code> , <code>macAddress</code> , <code>unixtime</code> e leituras não calibradas). JSONB permite consultar campos internos.
estacao_uuid	UUID	opcional	Copiado do payload para rastrear mensagens de uma estação sem abrir o JSON.
mac_address	VARCHAR(20)	opcional	Endereço físico do dispositivo, para suporte e depuração.
unixtime	BIGINT	opcional	Momento da leitura informado pela estação.
processado	BOOLEAN	padrão: FALSE	Indica se a mensagem já virou medida calibrada.
criado_em	TIMESTAMPTZ	padrão: agora	Momento em que a mensagem chegou ao servidor.
processado_em	TIMESTAMPTZ	coerente com processado	Momento do processamento. Só pode existir quando <code>processado = TRUE</code> .

Coluna	Tipo	Regras	Para que serve
tentativas	SMALLINT	padrão: 0	Quantas vezes o processamento falhou. Após N falhas, a mensagem vai para a Dead Letter Queue.
ultimo_erro	TEXT	opcional	Mensagem do último erro, para investigação.

Índices: `idx_staging_pendentes` é parcial e guarda só as mensagens não processadas, então continua pequeno mesmo com a tabela cheia; `idx_staging_processados_antigos` serve à rotina de limpeza; `idx_staging_estacao` serve ao suporte, para rastrear as mensagens de uma estação.

Crescimento: a tabela tem alta rotatividade e depende da rotina `fn_limpar_staging` (seção 5). O script traz, comentada, uma versão particionada por mês para quando o volume passar de alguns milhões de linhas.

4.9 medidas_resumo_diario

Resumo diário por sensor. Permite relatórios e análises de vários anos sem varrer milhões de leituras.

Quem escreve: rotina `fn_consolidar_dia`

Quem lê: `analise-dados`, `servico-relatorios`

Coluna	Tipo	Regras	Para que serve
id	BIGSERIAL	PK sequencial	Identificador do resumo.
sensor_id	UUID	FK → sensores · obrigatório	Sensor resumido.
dia	DATE	obrigatório	Dia consolidado.
valor_minimo	NUMERIC(12,4)	obrigatório	Menor leitura do dia.
valor_maximo	NUMERIC(12,4)	obrigatório	Maior leitura do dia.
valor_medio	NUMERIC(12,4)	obrigatório	Média das leituras do dia.
total_leituras	INT	obrigatório	Quantidade de leituras consideradas. Ajuda a detectar dias com falha de coleta.

Unicidade: `uq_resumo_sensor_dia` garante um único resumo por sensor por dia; reconsolidar o mesmo dia atualiza a linha existente.

5. Gatilho, funções e views

Objeto	Tipo	Para que serve
pgcrypto	extensão	Fornece <code>gen_random_uuid()</code> , usada para gerar os UUIDs das chaves primárias.
<code>fn_valida_municipio_gestor</code> + <code>trg_valida_municipio_gestor</code>	função + gatilho	Antes de inserir ou alterar um usuário, recusa a operação se o papel for <code>GESTOR_PUBLICO</code> e o município estiver vazio.
<code>fn_limpar_staging(dias)</code>	função	Apaga mensagens de staging já processadas e mais antigas que a retenção (padrão: 7 dias). Mensagens pendentes nunca são apagadas, porque indicam falha a investigar.
<code>vw_saude_staging</code>	view	Mostra pendentes, processadas, pendentes com falha, a pendente mais antiga e o tamanho em disco. Se "pendentes" só cresce, o processamento não acompanha a ingestão.
<code>fn_consolidar_dia(dia)</code>	função	Calcula mínimo, máximo, média e total de leituras de cada sensor em um dia e grava em <code>medidas_resumo_diario</code> . Pode ser executada de novo sem duplicar.
<code>vw_tamanho_tabelas</code>	view	Lista linhas aproximadas, tamanho total, tamanho dos índices e últimas execuções do autovacuum de cada tabela.

6. Decisões de modelagem

Decisão	Por quê
UUID nas entidades de cadastro	Identificadores que não revelam volume nem ordem e podem ser gerados em qualquer serviço.
BIGSERIAL em medidas, alarmes e resumos	Tabelas grandes lidas em ordem cronológica indexam melhor com inteiro sequencial.
NUMERIC para valores e coordenadas	Precisão exata, sem os arredondamentos de ponto flutuante, o que importa em calibração e limiares.
BIGINT para unixtime	Evita o estouro de 2038 dos inteiros de 32 bits, relevante para um histórico sem data de expurgo.
TIMESTAMPTZ em todas as datas	Datas guardadas com fuso horário, sem ambiguidade entre servidores e estações.
JSONB no staging	Guarda o payload como chegou, aceita formatos diferentes de sensor e permite consultar campos internos.
Exclusão lógica + ON DELETE RESTRICT	Nenhum dado histórico é perdido por uma exclusão; inativar preserva medições e alarmes.
Índices parciais	Indexam só o subconjunto consultado (regras ativas, mensagens pendentes, contas ativas), mantendo os índices pequenos.
Script idempotente	Usa IF NOT EXISTS, CREATE OR REPLACE e ON CONFLICT: pode ser executado várias vezes sem erro nem duplicação.

7. Operação e crescimento

Dimensionamento com a frequência definida pelo cliente (uma leitura a cada 5 minutos por sensor, 288 por dia):

Cenário	Linhas por dia em medidas	Por ano	Em 5 anos
Piloto: 4 estações x 7 variáveis	8.064	≈ 2,9 milhões	≈ 14,7 milhões
Expansão: 20 estações x 7 variáveis	40.320	≈ 14,7 milhões	≈ 73,6 milhões

Com o índice `idx_medidas_sensor_tempo`, o piloto fica bem dentro do que o PostgreSQL atende sem particionamento. No cenário de expansão, vale avaliar o particionamento anual de medidas e o uso do resumo diário nos relatórios.

Rotina recomendada

Frequência	O que fazer
Diário	Executar <code>fn_limpar_staging(7)</code> e conferir <code>vw_saude_staging</code> .
Semanal	Procurar mensagens de staging com tentativas > 0 e não processadas (presas por falha).
Mensal	Consultar <code>vw_tamanho_tabelas</code> , comparar com o mês anterior e conferir se o autovacuum está rodando.
Anual	Avaliar a consolidação diária do ano anterior e revisar o dimensionamento.
Sempre	Backup automatizado. O staging pode ser perdido sem grande impacto; medidas, estacoes e usuarios não.

Como executar o script

```
psql -U postgres -d pulso_urbano -f modelo_completo_postgres.sql
```

O script é o modelo de referência do produto. Na aplicação, cada microserviço cria as próprias tabelas por meio das migrations do `node-pg-migrate`, conforme o Manual de Instalação.

8. Pontos a alinhar

O script reflete o modelo definido no início da Sprint 1. Os itens abaixo divergem de decisões posteriores do squad, do cliente ou do código atual e devem ser resolvidos numa próxima versão.

Item	No script	Decisão ou situação atual
estacoes.codigo	Único em todo o sistema.	Decidido: único por município, UNIQUE(codigo, municipio).
sensores.tipo	Lista fechada com PM10; sem Direção do Vento, Precipitação e Pressão Atmosférica.	PM10 saiu do MVP; as 7 variáveis do MVP precisam estar aceitas. O RF-01 pede aceitar novas grandezas sem alterar o schema.
regras_alerta.esta_ativa	Nome esta_ativa.	Decidido: esta_ativo. A API atual também usa os campos nome e severidade.
Níveis de severidade	Informativo, Alerta e Critico.	Aguardando o cliente: manter 3 níveis ou adotar as 5 faixas oficiais do IQAr.
medidas_resumo_diario	Sugere manter o dado bruto só dos últimos meses.	O cliente definiu retenção enquanto o projeto existir: o resumo complementa, mas medidas não é expurgada.
Cabeçalho do script	Cita saúde pública, opção por MongoDB e staging como US-04.	Saúde saiu do escopo; o staging é PostgreSQL + Redis; a ingestão é a US-11 (Sprint 2).